

Diffusion-Based Generation of Novel Pokémon with Conditional Modeling

Aaron Cyril John, Yugaank Kalia, Varen Maniktala

MSML 612

Github Repository: <https://github.com/AaronCJohn/MSML-612-Project>

Abstract

This work presents a conditional diffusion model for generating novel Pokémon-style artwork with controls for elemental type, visual style (pixelated sprites, Sugimori styles, or 3D renders), and previous-evolution references. We assemble a diffusion-ready training corpus of 6,373 labeled entries from merged public asset packs and implement a U-Net with 128x128 RGB image inputs, self-attention at coarse resolutions, and a dedicated previous-evolution image encoder. We document the data pipeline, model architecture, training setup, sampling procedure, and evaluation against an unconditional diffusion baseline. The report summarizes dataset curation and preprocessing, qualitative samples, quantitative results, and limitations.

1 Problem Statement

Pokémon-inspired character art must balance creativity with faithfulness to the overall aesthetic: dual elemental types and a consistent look-and-feel across the distinct official asset styles. The core problem in this project is whether a deep learning model can generate novel Pokémon-like designs that follow these patterns while remaining distinct from any species in the training set.

Our baseline is an unconditional diffusion model trained only on Pokémon images, with no labels, no evolution pairs, and no reference sprite. This model can learn the overall visual distribution, but it cannot control type or style. The objective of this project is to build a conditional diffusion model trained on a unified RGBA dataset that folds heterogeneous conditioning, including multi-hot types, stage, style, timestep, and optional previous-stage imagery, into a single control signal. The model supports multi-style training within one network and allows a clear comparison with the unconditional diffusion baseline in terms of sample quality and conditioning accuracy.

2 Related Work and Literature Review

Diffusion and score-based image models. Denoising diffusion probabilistic models generate images by learning to iteratively remove noise, providing a practical alternative to GANs for high-fidelity synthesis [1]. Follow-on work improved stability and sample quality through better noise schedules, parameterizations, and sampling procedures [2]. Diffusion models also support conditioning through guidance mechanisms, which can improve adherence to inputs at inference time [3]. For higher resolutions, latent-space diffusion reduces compute by operating in a learned compressed representation [4], and alternative objectives broaden the design space of diffusion training [5]. We use classifier-free guidance to strengthen conditional generation at sampling time [6].

Backbones, conditioning, and image-to-image diffusion. Convolutional U-Net encoders and decoders remain the standard spatial backbone for pixel-space diffusion. Feature-wise conditional mechanisms (e.g., injecting label embeddings into intermediate feature maps) provide a lightweight way to incorporate global control signals into convolutional networks. In our model, conditioning is injected through the U-Net rather than only at the bottleneck. For paired image-to-image settings, Palette-style diffusion formulations translate between domains with diffusion

losses [7]. However, our final goal is attribute-conditioned generation rather than deterministic source-to-target replay alone. Conditional GANs such as pix2pix remain relevant baselines for paired translation quality.

Pokemon generation baselines. Prior Pokémon-generation projects primarily target standalone Pokémon-like images rather than controlled generation. Generated-Pokémon trains an unconditional DDPM/DDIM model on 819 JPG images and does not condition on type or style. Wong’s CS230 Pokemon generator uses StyleGAN2-ADA and a separate name-generation component, but it does not use a diffusion objective and does not provide attribute controls. In contrast, our contribution is a structured, controllable dataset and model intended to support attribute-conditioned generation.

Our approach in context. Our implementation follows public DDPM and U-Net practice, draws on improved diffusion and guidance ideas, and uses global label conditioning. Domain-specific contributions include the merged multi-style JSON dataset interface, dual-type multi-hot encoding, explicit masking for missing prior-stage images, and style/stage/type-conditioned sampling across sprite, Sugimori, and 3D domains.

2.1 Professor’s Notes Post Presentation: Related Projects

The following three projects were highlighted by Professor Ayhan after our presentation as closely related work. We briefly review each and articulate how our contribution extends or differs from the approaches taken.

Generated-Pokemon (Followb1ind1y, 2023). <https://github.com/Followb1ind1y/Generated-Pokemon>

This project trains both a DDPM and a DDIM sampler directly on the 819-image Kaggle Pokemon dataset (white-background JPGs). The model architecture follows the standard pixel-space U-Net backbone and does not incorporate any conditioning signal, so generation is entirely unconditional. Because every image is treated as an i.i.d. sample, the model has no knowledge of Pokémon type or artistic style. Our work explicitly addresses these limitations: we supply a structured JSON dataset that encodes dual-type multi-hot vectors, style labels (sprite / Sugimori art / 3D render), allowing the same DDPM backbone to perform controllable, attribute-conditioned generation across three visual domains.

Pokemon-Diffusion (DevikalyanDas, 2023). <https://github.com/DevikalyanDas/Pokemon-Diffusion>

This repository provides two complementary implementations: a plain unconditional DDPM and a Classifier-Free Guidance (CFG) variant conditioned on Pokémon type labels, which matches the classifier-free guidance formulation we also adopt [6]. The CFG model is trained on a type-annotated dataset and demonstrates qualitatively that conditioning on a single discrete class meaningfully steers generation toward type-appropriate visual features. Relative to this work, our pipeline extends conditioning beyond type to include a one-hot visual style token, enabling controlled generation across sprite, Sugimori, and 3D render domains. Our quantitative evaluation (FID, IS, LPIPS) also provides a more rigorous comparison than the qualitative grids reported in this project.

Stable-Diffusion-Pokemon-en (svjack, 2022). <https://huggingface.co/svjack/Stable-Diffusion-Pokemon-en>

This Hugging Face model fine-tunes the Stable Diffusion latent diffusion model [4] on a Pokémon image–caption dataset, yielding a text-to-image pipeline capable of synthesizing Pokémon-style art from free-form English prompts. Because it builds on a billion-parameter foundation model pretrained on internet-scale data, it benefits from strong low-level texture priors and compositional understanding that small pixel-space DDPMs lack. However, the approach requires substantial compute to fine-tune and depends on a large external checkpoint, making it difficult to ablate individual conditioning components. Our project deliberately targets a from-scratch pixel-space DDPM trained only on Pokémon imagery, which allows us to study the effect of each conditioning signal (type and style) in isolation without confounding influences from a pre-trained vision–language encoder.

3 Dataset and Preprocessing

3.0.2 Table 1: Raw image sources

Dataset	Raw Images
SugimoriSprites	
PokeSprite	
ProjectPokemon	

We constructed a dataset that integrated three complementary visual sources and augmented each entry with structured Pokémon metadata:

- SugimoriSprites: official-style 2D Pokémon artwork used as high-quality reference art.
- PokeSprite: compact pixelated game-style sprite assets used as the primary target sprite representation.
- ProjectPokemon / related render assets: large 3D render images used as an additional visual source for cross-format alignment.

As of May 2026, there are currently 9 total generations of Pokémon with around 1025 total Pokémon (not including various regional/fighting forms, megas, gigamax, etc., which bring that total up to nearly 2000 Pokémon). Our datasets, except for the Sugimori, offered shiny variants along with each Pokémon image (hence the higher dataset sizes). Only the PokeSprite dataset lacked the latest generation installment (hence the lower folder count). Finally, as a design choice, Megas/Gigamaxes were removed from the training as we felt that they were too distinct from the base Pokémon, which would hurt our conditional evolution variables.

3.0.2 Table 2: Raw dataset summary

Source	Files	Folders	Role
SugimoriSprites	1,848	1,025	High-quality 2D reference art
PokeSprite	2,853	905	Game-style sprite (primary target domain)
ProjectPokemon 3D	2,799	1,025	Alternate visual domain for cross-format use

3.1 Diffusion-Ready Dataset

For the final conditional diffusion model, we constructed a unified generative dataset in which every entry is a target image and conditioning vector pair. In the case that a Pokémon doesn't have any evolutions, the previous would be null.

Example schema:

```
JSON
{
  "prev": "bulbasaur",
  "next": "ivysaur",
  "prev_sprite": "poke-data/PokeSprite/0001bulbasaur/bulbasaur.png",
  "next_sprite": "poke-data/PokeSprite/0002ivysaur/ivysaur.png",
  "types": ["grass", "poison"],
  "art_style": "sprite"
}
```

3.1.1 Table 3: Diffusion-ready mappings

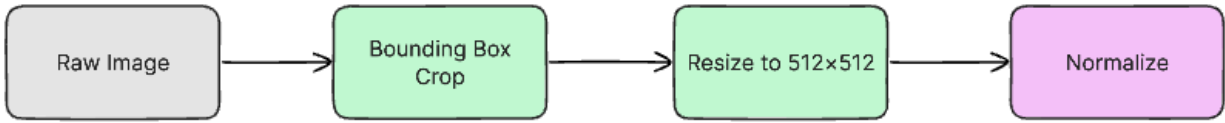
Mapping File	Entries	Role
sprite.json	1,995	Sprite to Sprite with previous-evolution image
sugimori.json	1,674	Sugimori to Sugimori with previous-evolution image
3d.json	2,704	3D to 3D with previous-evolution image
Total (merged in PokemonDiffusionDataset)	6,373	Unified diffusion training set

3.2 Curation Challenges

Building these mappings required solving problems that off-the-shelf datasets do not.

- Cross-format name collisions. The same Pokémon appears under incompatible filename conventions across sources, such as “0006 Charizard.png,” “charizard.png,” and “poke_capture_0006_*.png.” We resolved this through normalized lookup against poke-data/pokedex.json.
- Regional and cosmetic variants. Names such as “meowth-galar,” “bulbasaur shiny,” Mega, and Gigantamax share base Pokedex IDs. These suffixes are stripped and tagged so that type lookup stays unambiguous.
- Evolution-stage resolution. No single source labels the evolution stage directly. We derive stages by DFS over the previous-to-next edges from chain roots, with a PokeAPI fallback for entries not covered by our locally curated chains.
- Art-style labeling. Each mapping file is tagged with its visual domain, such as sprite, Sugimori, or 3D, so that a single unified dataset drives style-conditioned sampling at inference time.

3.3 Pre-processing Pipeline



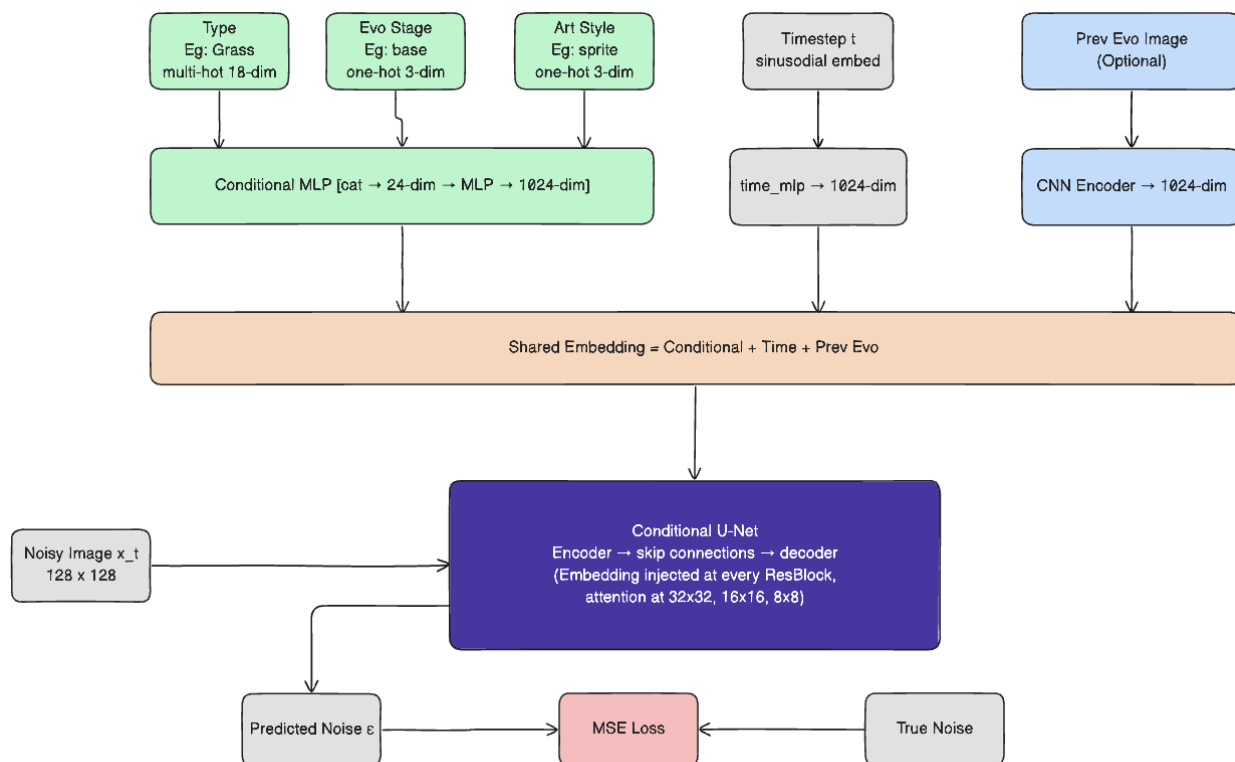
Upon completing filename normalization across datasets, JSON mappings (seen in Section 3.1) were created using type info and evolution chains from PokeAPI. Even though many of these mapping entries were created automatically, specific Pokémon that had inconsistent naming, including regional/fighting forms, had to be manually sorted and entered. This was one of the biggest slowdowns throughout the project, almost more so than training itself. Once all the JSON mappings were completed, we moved on to image preprocessing so that they would all have the same resolution and take up relatively the same amount of image frame. Since all obtained images are PNG, thus RGBA, we easily created a bounding box generation pipeline that utilized the transparent pixels as guidelines for creating Pokémon-dependent boxes. This was necessary as each dataset varied in resolution sizes (PokeSprite: 68x56, ProjectPokemon: 512x512, Sugimori: 100x100-1000x1000) and so standardization was essential for the diffusion model to accurately understand Pokémon dimensions.

After generating the bounding boxes for each image across datasets, the images were then resized and padded using those bounding boxes to center paste the Pokémon onto a 512x512 PNG image frame (512x512 was chosen since noise was added on smaller/larger resolutions) using the resampling methods Nearest (for sprites) and Lanczos (for 3D/Sugimori). Padding was used in conjunction with bounding boxes, since edge cases existed where some Pokémon would extend past the image frame due to inconsistent aspect ratios.

Finally, all images were resized to RGB 128x128 (after obtaining the correct transparency masking) so that sprites, Sugimori artwork, and 3D renders share one consistent tensor representation before training. Additionally, during training, random augmentations like horizontal flips and color jitter (across contrast and brightness) were applied to create nearly infinite variations of each image.

4 Model Architecture and Implementation Details

4.1 Architecture Overview



The model is a conditional U-Net with:

- Base channel width of 64, channel multipliers of 1, 2, 2, and 4, and 2 ResBlocks per level.
- Conditioning injected throughout the network via a shared conditioning embedding, computed once per sample and provided to each ResBlock.
- Self-attention at 32x32, 16x16, and 8x8 spatial resolutions to capture long-range shape coherence that pure convolutions miss.
- Dropout of 0.1 inside ResBlocks for regularization on a small dataset.
- Training at 128x128 RGB resolution.

4.2 Conditioning Design

Five conditioning signals of different types: type labels, evolution stage, art style, a reference image, and the diffusion timestep, all compressed into a single 128-dimensional control vector that is provided to the U-Net at multiple resolutions.

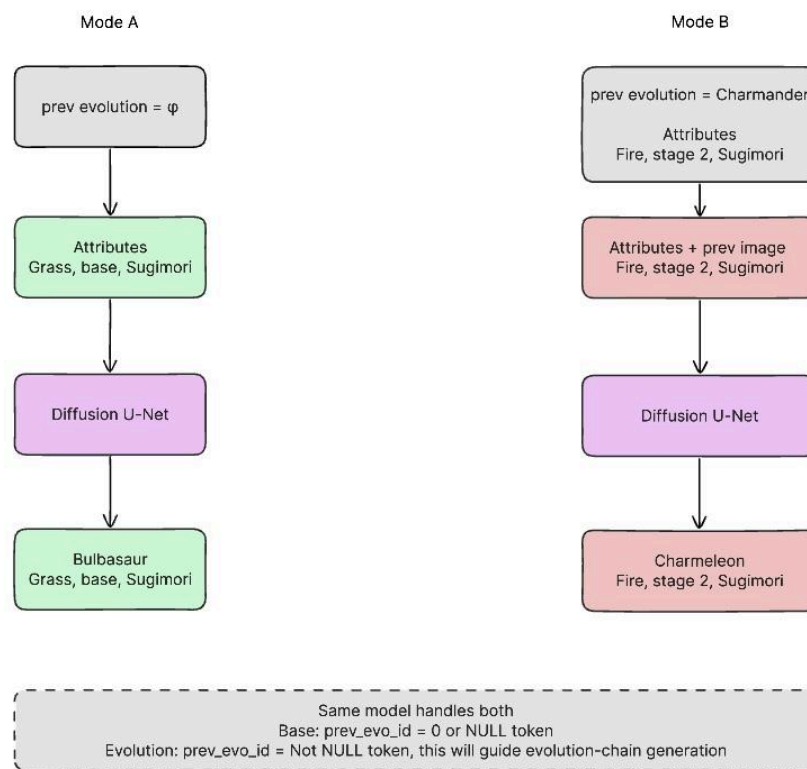
4.2.1 Table 4: Conditioning inputs

Conditioning Input	Encoding	Why this encoding
Pokemon Type (e.g., Fire, Water)	Multi-hot over 18 types (supports dual-type)	One-hot would lose dual-typing entirely
Evolution Stage (base / evo1 / evo2)	One-hot over 3 stages	Discrete structural complexity control

Visual Style (3d / Sugimori / sprite)	One-hot over 3 styles	Enables cross-domain sampling at inference
Previous Evolution Image	CNN encoder → 128-d vector (zeroed if absent)	Base-stage Pokémon have no prior; mask token keeps batch shape valid
Timestep t	Sinusoidal embedding → MLP	Standard DDPM timestep encoding

All embeddings are concatenated and fused by a `cond_combine` MLP into a single 128-dimensional vector. This vector is infused into the denoising network at multiple depths. This places the conditioning signal close to the evolving feature map rather than relying on a single injection point.

4.3 Training Procedure



4.3.1 Table 5: Training setup

Component	Setting
Objective	MSE between predicted noise and true noise (standard DDPM)
Optimizer	AdamW, lr = 2e−4, weight_decay = 1e−4
LR schedule	Cosine annealing over 500 epochs
Noise schedule	Linear beta from 1e−4 to 0.02 over 1000 timesteps
EMA	Decay = 0.9999 on model weights (used for sampling)
Regularization	Gradient clipping at 1.0, dropout 0.1 inside ResBlocks

Batch size	64
Image size	128×128 RGB
Base channels / multipliers / ResBlocks	64 / (1, 2, 2, 4) / 2 ResBlocks per level
Attention resolutions	32×32 , 16×16 , 8×8
Hardware	Google Colab G4 High-RAM GPU: NVIDIA RTX PRO 6000 Blackwell

4.3.1 Table 6: Training outcome

Component	Value
Parameters	Approximately 22M (dependent on architecture)
Epochs completed	500
Wall-clock training time	Approximately 5 hours (dependent on architecture)
Training curve	See Figure 1
Hardware	Google Colab G4 High-RAM GPU: NVIDIA RTX PRO 6000 Blackwell

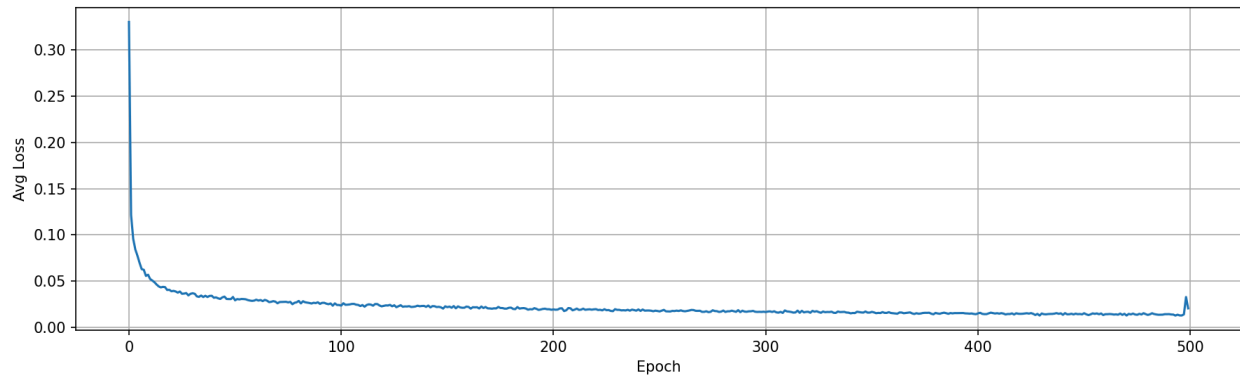


Figure 1: Training loss over 500 epochs for the unconditional diffusion baseline.

4.4 Software Layout

Training and evaluation are driven from a single `main.py` file that utilizes separate train and inference Python files, each with configuration files (for architecture and where data is loaded from). Add a single `PokemonDiffusionDataset` loader over the three merged JSON mapping files, and version-controlled train/validation index JSON files for repeatable splits. Checkpoints store both raw and EMA weights and support resume functionality. Sampling exposes flags for type, stage, and style.

5 Evaluation Results

5.1 Qualitative Results

5.1.1 Unconditional diffusion samples



Figure 2: Unconditional diffusion samples from the image-only baseline

The unconditional diffusion baseline generated Pokémon-like samples from image-only training. These samples captured some broad visual patterns but lacked controllability because the model had no access to information about type or style. 3D renders, while blobby, do exhibit some consistent coloring; sprites, on the other hand, have consistent structure (black pixel outline) and even cohesive interior detail. We decided not to train a Sugimori baseline as we found that our model struggled the most with these kinds of samples (consistently blobbier than 3D).

5.1.2 Type-conditioned samples



Figure 4: Style-conditioned samples spanning 3D, Sugimori, and sprite outputs.

The type-conditioned diffusion model generated samples in both 3D and sprite domains. These results demonstrate that the model can use type information as part of the conditioning signal, although visual consistency still depends on the style and training domain. The 3D samples (on the left of Figure 4) were generated with the following configs, respectively (top to bottom): Type: Psychic, Evo: base; Type: Grass/Poison, Evo: base; Type: Dragon, Evo: 1; Type: normal, Evo: 2. The sprite samples (on the right of Figure 4) were generated with the following configs, respectively (top to bottom): Type: Water/Flying, Evo: 1; Type: Grass/Poison, Evo: base; Type: Dragon, Evo: 1; Type: normal, Evo: 2. Noticeably, there is much more structure across samples compared to the baselines. Some images, particularly the first 3D sample on the 3rd row, resemble actual Pokémon like Altaria. Additionally, the first sprite image in the top row does seem to follow the conditions of being a water flying Pokémon (hence the blue wing-like structures).

5.1.3 Style-conditioned samples



Figure 4: Style-conditioned samples spanning 3D, Sugimori, and sprite outputs.

Style-conditioned sampling demonstrated the model’s ability to generate across 3D (top row), Sugimori (middle row), and sprite outputs (bottom row). This confirms that the unified dataset and style labels allow one network to learn multiple visual domains. The 3D renders were generated as rock base, steel base, and water dragon evo 2 (this image in particular resembles a mix of Lapras and Kingdra), respectively. The Sugimori were generated as water base (this image in particular resembles a mixture of Magikarp and Tatsugiri), steel base, and rock base. Finally, the sprites generated as poison base (resembles a mixture of Nidoran (male) and Noibat), ice evo 2, and water ground base. Again, one can easily see that our model was the most consistent in creating pixelated sprites, most likely because the sprites don’t have that fine detail that our model struggles to mimic (additionally, the consistent black pixel outline allows the model to pick up on common patterns).

5.2 Quantitative Evaluation, Runtime, and Ablations

5.2.1 Table 7: Evaluation metrics

Metric	Description	
FID (Frechet Inception Distance)	Distribution distance between real and generated images; lower is better	Realism and distribution match
KID (Kernel Inception Distance)	FID-like distribution metric that is less biased for smaller sample counts	Small-sample realism and diversity
ID (Intra-class Distance)	Average pairwise distance across generated samples	Sample diversity
Qualitative Review	Side-by-side visual inspection with attribute adherence	Conditioning adherence

5.2.2 Table 8: Quantitative comparison

Metric	Unconditional Diffusion Baseline	Conditional Diffusion (Single Style)	Conditional Diffusion (Multiple Styles)
FID (lower is better)	123	77	202
KID (lower is better)	0.132 ± 0.002	0.061 ± 0.0017	0.231 ± 0.033
ID (higher is better)	15.44	16.23	17.39

The single-style conditional model improves both FID and KID over the unconditional diffusion baseline, while the multi-style model increases the diversity-oriented ID score at the cost of a weaker distributional match. This matches the qualitative observation that multi-style training is harder because one network must cover sprite, Sugimori, and 3D visual domains at once. In practice, much of the degradation came from the Sugimori set: the samples weren't able to capture the detail and texture of the Sugimori art style, leading to a disproportionate increase in pooled FID and KID relative to sprite and 3D branches.

eval.py evaluates the diffusion model checkpoints by generating images and comparing them against real Pokémon images. For each checkpoint, it generates N samples (default 2000) via DDIM sampling using null conditioning for conditional models and then passes both real and generated images through a pretrained Inception V3 network to extract 2048-dimensional pool-layer features. From these features it computes three metrics: FID (Fréchet Inception Distance), which measures the distance between the real and generated feature distributions by comparing their means and covariances (lower is better), KID (Kernel Inception Distance), which uses a polynomial kernel MMD estimator over random subsets for a more robust distributional comparison (lower is better) and ID (Intra-Class Distance), the mean pairwise Euclidean distance among generated features, which quantifies output diversity (higher is better).

5.2.3 Table 9: Runtime and Efficiency

Measurement	Unconditional Diffusion Baseline	Conditional Diffusion
Parameters	Approximately 19M	Approximately 24M
Training hardware	Google Colab G4 High-RAM GPU	Google Colab G4 High-RAM GPU
Wall-clock training	Approximately 5 hours for 500 epochs	Approximately 5 hours for 500 epochs

time

Sampling steps	1000 DDPM steps; DDIM used for faster previews	1000 DDPM steps; DDIM used for faster previews
Input resolution	128 × 128 RGB	128 × 128 RGB

5.2.4 Table 10: Ablation Study

Experiment	What it tests	FID	Notes
Unconditional vs. attribute-conditioned	Effect of type/stage/style on quality	123 vs. 77	Single-style conditioning improves FID
Single-style vs. multi-style joint training	Cross-domain transfer	77 vs. 202	Multi-style training increases task difficulty
Unconditional vs. multi-style conditional	Diversity under broader conditioning	123 vs. 202	Multi-style model has higher ID but worse FID
EMA weights at sampling	Sample stability	Used	EMA checkpoint used for stable samples

These comparisons isolate the effects of attribute conditioning, style scope, and EMA sampling. We restrict quantitative ablations to type and style signals because they are consistently available across the unified dataset.

6 Insights, Limitations, and Future Scope

6.1 Insights

What worked well: The unified multi-source dataset of about 6.4K entries with clean types, stage, and style labels made it possible to train a single model that generalizes across sprite, Sugimori, and 3D styles. Conditioning integrates cleanly with the U-Net, with no architectural surprises. The single-style conditional model achieved better FID and KID than the unconditional diffusion baseline, showing that structured labels are useful rather than only decorative.

What was hard: Cross-format name normalization, including shiny, mega, gigantamax, and regional variants, required careful hand-written rules. Relying on filenames alone was insufficient. Evolution-stage resolution had to be implemented with DFS over previous-to-next edges, plus a PokeAPI fallback because no single source labels stage directly. Preprocessing also took a large fraction of the project because naming, orientations, and visual structures were inconsistent across art styles. Finally, the dataset is small relative to typical diffusion training runs, which made careful augmentation and the EMA model copy important for stable sampling.

Takeaway: Relative to the unconditional diffusion baseline, the conditional diffusion formulation is a better match for a structured, low-data, multi-domain Pokémon setting. Iterative denoising remains stable, attribute conditioning is integrated at every residual block through a 128-dim control vector, and one model can be trained over all three visual styles.

6.2 Limitations

- Trained at 128x128 RGB, so fine sprite detail is still bounded by resolution.

- Only 3 styles and 3 evolution stages are modeled. Real Pokémon have far more visual variants, such as shiny, mega, and regional forms, that we do not yet condition on explicitly.
- There is no text conditioning, so the model cannot follow natural-language prompts describing abilities or lore.
- The code supports evolution-chain sampling, but the generated chains do not yet maintain enough visual coherence across stages.

6.3 Future Scope

- Scale up resolution with a latent diffusion or super-resolution stage.
- Use classifier-free guidance through conditioning dropout for stronger attribute adherence at sampling time.
- Add text conditioning using a pretrained text encoder such as CLIP for descriptive generation.
- Develop a full evolution-chain consistency loss that ties together all generated stages jointly rather than one stage at a time.
- Use RGBA masking or an alpha-aware post-processing stage to recover cleaner transparent sprite outputs from RGB generation.

7 Contribution of Each Team Member

Aaron Cyril John: Verified the curated datasets and cross-checked asset mappings so that paired entries across formats aligned correctly and inconsistencies were resolved before training. Prepared all written reports and presentation slides so documentation matched the final pipeline and results. Participated in model training runs and inference alongside the rest of the team.

Yugaank Kalia: Developed the model architecture, training pipeline, and the JSON dataset files required to train the conditional diffusion model. Additionally, developed mapping scripts that link Pokémon to their previous evolutions and other necessary conditioning variables.

Varen Maniktala: Developed the dataset preparation and curation/preprocessing pipeline. Additionally, created scrapers/sorters code that downloaded all images from the various sources (Bulbapedia/GitHub/Zips) and placed them all in dedicated Pokémon/art style folders. Helped with training and ensuring correctness along the training pipeline. Debugged issues in the model architecture and assisted in hyperparameter tuning.

8 Tools and Libraries Used

8.1 Core Libraries and Stack

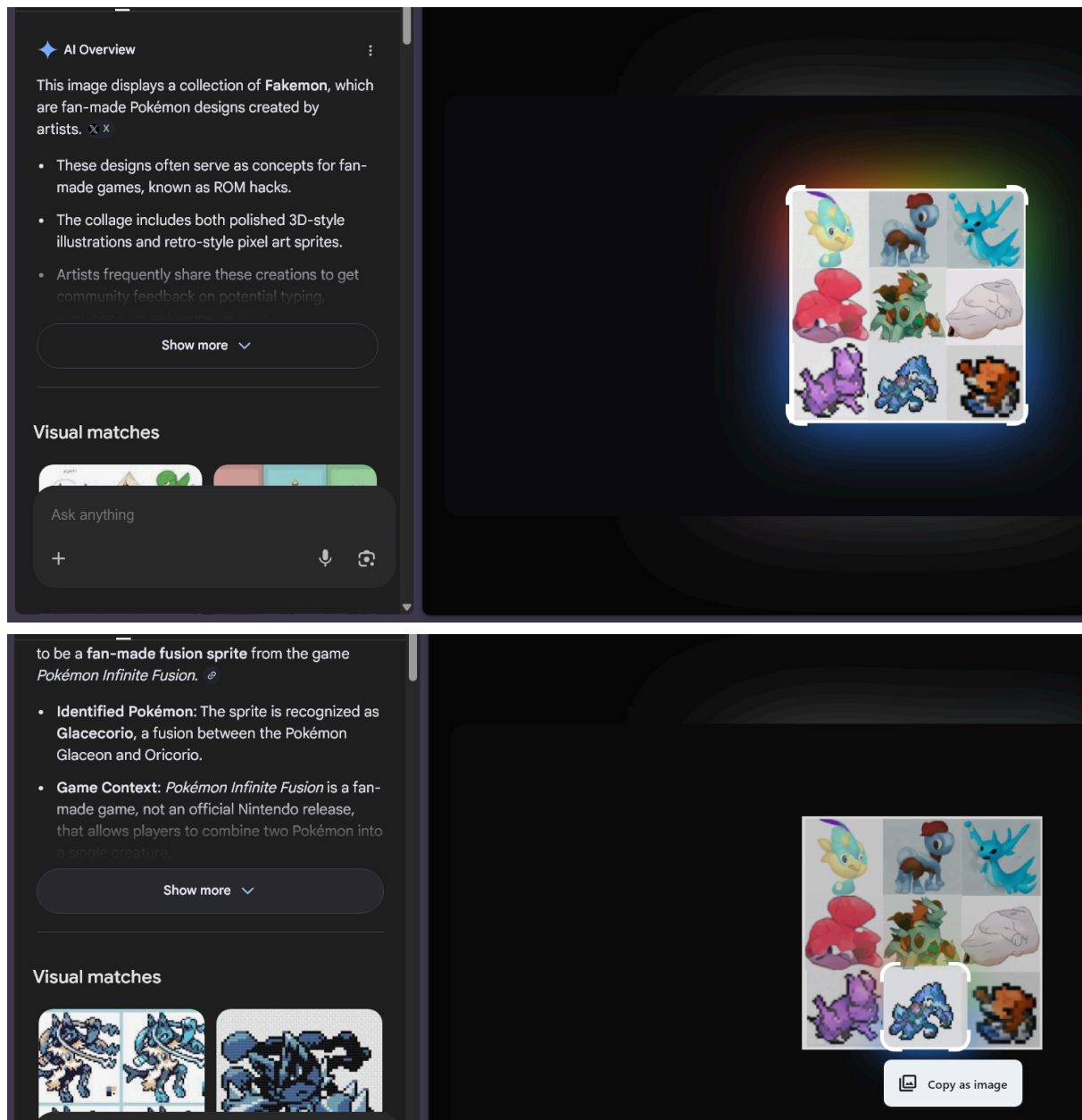
The implementation is written in Python 3 with PyTorch and torchvision for tensor computation, optimizers, and image I/O/transforms. Numerical utilities rely on NumPy. Plotting and contact-sheet figures for curation use Matplotlib where applicable. Optional preprocessing utilities in the repository may additionally use Pillow. External metadata is supplemented through the PokeAPI. Public references for diffusion tooling include the Hugging Face Diffusers library as related reading, though the course model code is implemented directly in PyTorch. Instructions on reproducibility are included in the [README.md](#) file in the github repository.

A References

- [1] Ho, J., Jain, A., and Abbeel, P. Denoising Diffusion Probabilistic Models. NeurIPS, 2020. <https://arxiv.org/abs/2006.11239>
- [2] Nichol, A., and Dhariwal, P. Improved Denoising Diffusion Probabilistic Models. ICML, 2021. <https://arxiv.org/abs/2102.09672>
- [3] Dhariwal, P., and Nichol, A. Diffusion Models Beat GANs on Image Synthesis. NeurIPS, 2021. <https://arxiv.org/abs/2105.05233>
- [4] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-Resolution Image Synthesis with Latent Diffusion Models. CVPR, 2022. <https://arxiv.org/abs/2112.10752>
- [5] Karras, T., Aittala, M., Aila, T., and Laine, S. Elucidating the Design Space of Diffusion-Based Generative Models. NeurIPS, 2022. <https://arxiv.org/abs/2206.00364>
- [6] Ho, J., and Salimans, T. Classifier-Free Diffusion Guidance. NeurIPS Workshop, 2021. <https://arxiv.org/abs/2207.12598>
- [7] Saharia, C., Chan, W., Saxena, S., et al. Palette: Image-to-Image Diffusion Models. SIGGRAPH, 2022. <https://arxiv.org/abs/2111.05826>
- [8] Ronneberger, O., Fischer, P., and Brox, T. U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI, 2015. <https://arxiv.org/abs/1505.04597>
- [9] Isola, P., Zhu, J.-Y., Zhou, T., and Efros, A. A. Image-to-Image Translation with Conditional Adversarial Networks (pix2pix). CVPR, 2017. <https://arxiv.org/abs/1611.07004>
- [10] Hugging Face8. <https://arxiv.org/abs/1709.07871>. Diffusers: State-of-the-art diffusion models for image and audio generation in PyTorch. <https://github.com/huggingface/diffusers>
- [11] PokeAPI. <https://pokeapi.co/>
- [12] msikma. pokesprite. <https://github.com/msikma/pokesprite>
- [13] Project Pokemon. Sprite and asset resources. <https://projectpokemon.org/>
- [14] Pokemon Database. National Pokedex and image resources. <https://pokemondb.net/pokedex/national>
- [15] Followb1ind1y. Generated-Pokemon: New Pokemon Generation using Diffusion Models. <https://github.com/Followb1ind1y/Generated-Pokemon>
- [16] Wong, R. Pokemon Generator. Stanford CS230, 2021. https://cs230.stanford.edu/projects_fall_2021/reports/103146257.pdf
- [17] DevikalyanDas. Pokemon-Diffusion. <https://github.com/DevikalyanDas/Pokemon-Diffusion>
- [18] svjack. Stable-Diffusion-Pokemon-en. <https://huggingface.co/svjack/Stable-Diffusion-Pokemon-en>

B Supplementary tables and figures

The images below are from an experiment with Google's Visual Search to see how Gemini reacted to specific generated Pokémon images. Since Gemini classified these images as either Fakemon or even actual Pokémon Fusion creatures, it shows that these generations do, in fact, follow some of the styling required by real Pokémon.



AI Overview

This image appears to show a pixel art sprite, specifically a fusion of Pokemon characters.  Etsy +1

- The visual style is consistent with retro game graphics.
- The sprite design looks like a handmade or fan-created fusion, often found in projects recreating Pokedex entries.
- Similar pixel art designs are used in DIY crafts,

Show more ▾

Visual matches



 Copy as image